Group X Progress Report:
My Group's Project Name

Bhavya Patel, Anaum Sajjad, Third Author
{patelb61,macid2,macid3}@mcmaster.ca

1 Introduction

Myocardial Infarction, commonly known as a heart attack, may lead to complications and other outcomes. There is a need to be attentive to the early identification of vulnerable patients who may develop complications. It could be incredibly difficult to track how vulnerable someone is to complications because there are other underlying causes such as age, race, medical history, symptoms, and clinical measurements.

We use the Myocardial Infarction Complications dataset from the UCI Machine Learning Repository. This dataset contains 1,700 patient records, 111 input features, and 12 outcome variables (Golovenkin et al., 2020). The prediction task contains 11 binary complications and 1 eight-class categorical outcome.

Our model focuses on the clinical information available after 72 hours, model focuses on clinical information available through the first 48 hours of hospitilization. Input features will reduce to 108 allowing us to investigate trade-offs between earlier prediction and predictive performance.

2 Related Work

There has been research aimed to developing stratification for the mortality of patients with acute myocardial infarction. (Kwon et al., 2019) investigated mortality prediction using the data found from 22,875 patients, this demonstrates the potential of deep learning for health risk prediciton.

Yavru and Yılmaz Gündüz (2022) developed a deep learning model using hidden layers, dropout, and skip connections to predict myocardial infarction complications. Investigated training on both original and augmented data, reporting an overall binary accuracy of 92

Multi-task learning is particularly relevant to our approach since it would allow models to learn multiple related prediction tasks at the same time. (Caruana , 1997) demonstrated that multi-task learning improves generalization by sharing information between related tasks through common representations, motivating the MLP.

3 Dataset

We are using the publicly available Myocardial Infarction Complications dataset from the UCI Machine Learning Repository. As the dataset is annotated, no additional annotation was done. The dataset contains 1700 patient records with 111 features and 12 target columns. The input features include patient demographics along with clinical measurements and information. The 12 target variables consist of 11 binary outcomes and one 8-class categorical outcome.

3.1 Feature Selection and Missing Data

During the initial data analysis, columns corresponding to information available after 48 hours of hospitalization (`R_AB_3_n`, `NA_R_3_n`, and `NOT_NA_3_n`) were dropped, as it did not align with our goal. The missing data distributions for each feature were analyzed and anything with over 50% missing data was examined closely. Features `KFK_BLOOD` and `IBS_NASL` were dropped immediately as they had 99.76% and 95.76% missing values, respectively.

Two features representing emergency response blood pressure measurements (`D_AD_KBRIG` and `S_AD_KBRIG`) had 63.29% missing values. Correlation analysis was run between these columns and their corresponding ICU blood pressure measurements: the correlation between `D_AD_KBRIG` and `D_AD_ORIT` was 0.56 while the correlation between `S_AD_KBRIG` vs `S_AD_ORIT` was 0.61. They were not strongly correlated enough to justify removal. A missing analysis was conducted and both columns had missing values for the same instances, so one binary indicator column was cre-

ated (EMR_RES_BLOOD) to represent both columns. EMR_RES_BLOOD would have a value of 1 if ER blood pressure measurement was taken or 0 if not taken.

After feature selection was complete, there were 107 input features remaining. These features were analyzed and divided into these categories: numerical, binary, ordinal or partially ordered to determine how to appropriately preprocess them. A correlation analysis was run for the numerical features, however most of the relationships ranged from weak to moderate, and therefore no further features were dropped.

3.2 Target Distribution and Data Splitting

Although the target columns contained no missing values, there was a large class imbalance. Most binary target columns had above 1600 "no" instances and under 100 "yes" instances. The categorical target column "LET_IS" had categories 6 and 7 representing under 2% of the outcomes. To preserve the distribution of possible outcomes across the train, test and validation sets, the split was done using iterative stratification. As iterative stratification required binary labels, the categorical target column LET_IS was temporarily one-hot encoded into 8 columns. The indices found in the split were then applied to do the split on the original dataset to allow LET_IS to remain a categorical column. The dataset was divided into 70% train, 15% validation and 15% test.

3.3 Missing Value Imputation and Feature Scaling

Missing value imputation and feature scaling were done after the split to prevent data leakage. The numerical features were imputed using train medians as they are less sensitive to outliers. The categorical columns (binary, ordinal and partially-ordered) columns were imputed using the train modes. These imputation values were then applied to test and validation sets.

For feature scaling, similarly, the scalers were fitted to the train set before being applied to the validation and test sets. The numerical features were scaled using `StandardScaler`. The ordinal features and the partially ordered feature were scaled using `MinMaxScaler` to map values between 0 and 1. The binary columns were left as is. The resulting datasets were named: `X_train`, `y_train`, `X_validation`, `y_validation`, `X_test`, `y_test`.

3.4 Future Considerations

In the future, the partially ordered column ZSN_A might be investigated further as it has 2 severity branches and therefore does not follow a single ordinal ranking, but currently it has been left in its original form.

4 Features

The multitask MLP receives the preprocessed clinical features as input. Missing values were imputed, and numerical features were scaled before converting them into numerical arrays for model training. Representation learning was performed using two shared hidden layers. The first hidden layer has 64 neurons and the second has 32, and both use ReLU as the activation function. These layers learned non-linear combinations of the input features and are used to predict two separate output branches, which are the binary complications and the multiclass LET_IS outcome. Both prediction tasks are part of a single neural network, which allows them to share the learned representation. Feature selection was used to remove variables based on their availability during the observation window and according to how many of their values were missing. No separate embedding layers or data augmentation techniques were used.

5 Implementation

5.0.1 Model Architecture

We implemented a multitask Multilayer Perceptron (MLP) using TensorFlow and the Keras API to predict myocardial infarction complications and outcomes. The preprocessed clinical patient features are provided as input to the model, which consists of two shared fully connected hidden layers. The model layers are discussed in Section 4. The neural network contains two output branches.The first branch predicts the probability of each binary complication, where the branch consists of 11 neurons with sigmoid as the activation function. The second branch predicts the categorical LET_IS outcome, where the branch consists of 8 neurons with softmax as the activation function.

5.0.2 Loss Function

There is significant class imbalance in the dataset. To address this problem, we use class-weighted cross-entropy loss functions. Without weighted loss functions, the model tends to predict mostly

negative outcomes for binary complications and the majority class for LET_IS.

The positive class weights for binary complications are calculated using:

$$w^{+} = \left(\frac{N_{\text{negative}}}{N_{\text{positive}}} \right)^{\alpha} \tag{1}$$

where N_j^- represents the number of negative samples and N_j^+ represents the number of positive samples for complication j.

For binary complications, we use the Binary Weighted Cross-Entropy:

$$\mathcal{L}_{\text{binary}} = -\left[y \log(p) + (1-y) \log(1-p) \right] \tag{2}$$

where $y_j \in \{0, 1\}$ is the true label for complication j, and p_j is its predicted probability. A label of 0 indicates that the complication does not occur, while a label of 1 indicates that it does occur.

For the multiclass outcome, we use the Weighted Sparse Categorical Cross-Entropy:

$$\mathcal{L}_{\text{multiclass}} = -w_y \log(p_y) \tag{3}$$

where w_y represents the weight assigned to the true class y, and p_y is the predicted probability of that class.

The model combines the two losses using:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{binary}} + \mathcal{L}_{\text{multiclass}} \tag{4}$$

The Adam optimizer minimizes the combined loss by updating the shared hidden layers and the two output branches.

We experimented with three weighting strategies, which were full inverse-frequency weighting, square-root weighting and weighting with alpha=0.7, which is the intermediate of the previous two. Full weighting (alpha=1.0) caused the model to predict a large number of positive complications, but also predicted a huge number of false positive complications. The square root weighting (alpha=0.5) only predicted a few true positive complications, causing an increase in the prediction of false negatives. The weighting with alpha=0.7 is intermediate between the previous two weighted loss functions. This approach was able to predict more True positives and fewer false negatives, but more false positives.

5.0.3 Optimization Technique

The Adam optimizer was used to train the model with a learning rate of 0.001 and a batch size of 32. Adam was selected because it adaptively adjusts parameter updates using the first and second moments of gradient estimates.

TensorFlow performs automatic differentiation and backpropagation to calculate the gradients, which Adam then uses to update the neural network parameters.

The model was trained for a maximum of 50 epochs with early stopping. Early stopping monitors the validation loss and stops training when the validation loss does not improve for 5 consecutive epochs. The model weights are then restored from the epoch with the lowest validation loss. This approach helps reduce unnecessary training and overfitting.

Precision and recall were used as training metrics for binary complication prediction, with a classification threshold of 0.5. Accuracy alone was not selected because it can be misleading when the majority of target labels are negative.

Since the LET_IS output represents a multiclass classification problem with 8 possible classes, sparse categorical accuracy was used because the target labels are stored as integer class indices rather than one-hot encoded vectors.

6 Results and Evaluation

6.0.1 Results

The model with the full class weighting increased the number of positive predictions significantly. For instance, the FIBR_PREDS contains 26 positive cases, and the model predicted 92 positive cases; hence, this type of weighting caused overprediction. The validation loss for this model barely decreased over many epochs, where its initial validation loss was 3.423, and the final was 3.289. The Square-root weighting had the opposite effect, with a decreased number of positive complication predictions. For instance, the FIBR_PREDS contains 26 positive cases, and the model predicted only 1 positive case. The validation loss over many epochs was comparatively lower than the full weighting model, from initial being 1.856 and the final loss being 1.466. The intermediate weighting with alpha=0.7 had a relatively more balanced result when compared to the previous two models. For instance, the FIBR_PREDS contains 26 positive cases, and the model predicted only 13 positive cases. The

validation loss was lower than the full weighting model but higher than the square-root model, from an initial value of 2.255 to a final value of 2.007. These experiments provided a basis for further investigation of loss weighting and model generalization.

6.0.2 Baseline Comparison

In the baseline model, the majority class is 0 for the 11 binary complications, so the baseline always predicts the patient will not experience the complication. The majority class is 0 for the LET_IS, so the baseline always predicts class 0. We created another Jupyter notebook that takes all the metrics for all the models and compares them to evaluate which model performed better. The square-root weighting model had the highest mean macro F1-score of 0.4887, when compared to the baseline with 0.4807. But the square-root model did not perform well for predicting the positive complication, and instead the full-weighting model performed better with 0.5523, when compared to the baseline with 0.500. But at the same time, the full-weighting model had a lower macro F1 score and accuracy when compared to the baseline. The square-root model for the multiclass LET_IS outcome performed the best compared to every model, where the macro F1 score was 0.2511, the accuracy was 0.8588, and the macro precision was 0.3210. However, the full-weighting model had a better macro recall of 0.2981. The intermediate model had the lowest accuracy for both binary complications and LET_IS outcome, with an accuracy of 0.5533 and 0.0392, respectively. It had a lower score than the baseline model, which calls for further investigation. Overall, the square-root model performed better when we consider the macro F1-score, but it also struggles with the imbalanced data.

7 Evaluation Strategy

To evaluate the multi-task learning model, there is a division of the preprocessed dataset for Myocardial Infarction Complications. The data is split into 3 sets: training, validation, and testing, in a 70/15/15 split. Within the training set, there are 1,190 patient sets; our validation and testing sets contain 255 records each. There are 107 input features and 12 outcomes (11 binary complications and 1 eight-class categorical outcome).

Here, we are training a model; the training set is used to take these data points and train the model to see what it can do; validation is used primarily to help monitor the performance and compare the model configurations. Lastly, testing is crucial to ensure the chosen model meets standards. In our implementation, we used a fixed train-validation split instead of k-fold cross-validation.

The evaluation strategy will focus on 4 main things: accuracy, precision, recall, and F1 score for binary complication outcomes. Many complications will occur infrequently; accuracy by itself could be too low to represent a model's ability to identify patients at risk. Looking at specific complications like the `PREDS_TAH` complication there are only 14 positive cases within the training set, there are 3 in validation and then lastly 3 in testing set.

A confusion matrix is used to examine all the false negatives and false positives in a dataset; the `LET_IS` categorical outcome is used to evaluate a model's prediction. The model is compared against a majority-class baseline to see whether the multi-task learning approach provides improvement over predicting the most common outcome.

8 Feedback and Plans

The TA (Shreya) recommended comparing our current MLP using 48-hour features with the same architecture using 72-hour features to see the impact of reduced input features. She also suggested experimenting with L1 and L2 regularization to reduce overfitting, exploring other models, and including all project files with a detailed README.

Our initial experiments addressed class imbalance using full inverse-frequency, square-root weighting, and weighting with an exponent of 0.7. Full weighting produced many false positives (127 for `PREDS_TAH` and 0 true positives), while square-root weighting predicted almost all negatives. All models struggled with rare complications. The full weighting model also showed a lower training loss (under 2) than validation loss (around 3.3), suggesting possible overfitting.

Moving forward, we plan to evaluate binary targets using precision, recall, F1-score, and precision-recall AUC as well as doing performance evaluation for each class in the `LET_IS` target column. We will investigate L1/L2 regularization, alternative architectures, and cross-validation due to the low number of positive outcomes. After selecting the best configuration using validation results, we will evaluate it on the test set. Finally, we will

compare the 48-hour and 72-hour feature sets to determine whether complications can be predicted using only information available within the first 48 hours of hospitalization which addresses our main research question.

Team Contributions

Bhavya Patel worked on intial data analysis and the data preprocessing. She wrote sections: "Dataset" and "Feedback and Plans". Anaum Sajjad worked on the model implementation. She write the sections: "Features", "Implementation", and "Results and Evaluation".

References